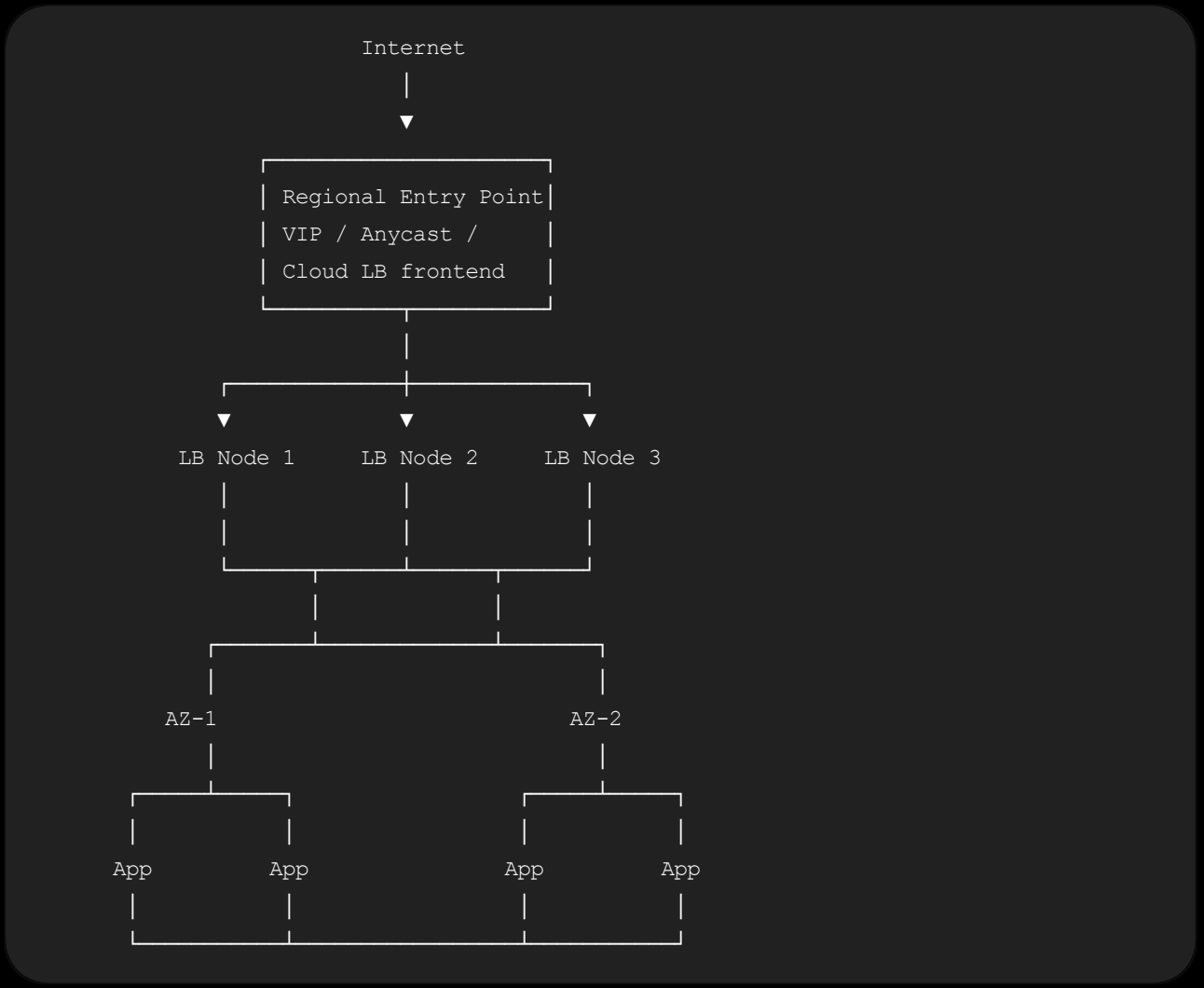
Yes. If **all 1M requests/sec are going to one region**, you don't solve the problem by buying a bigger LB VM. You build a **distributed load-balancing tier inside that region**.

The key is to separate **the logical load balancer** from **the physical LB instances/nodes**.

1M RPS architecture



But there are some important subtleties.

---

1. First calculate what 1M RPS actually means

1M RPS alone isn't enough to size the LB.

For example:

Scenario A — tiny requests

```
1M requests/sec
× 1 KB
≈ 1 GB/sec
≈ 8 Gbps
```

That's significant but manageable with a distributed fleet.

Scenario B — 100 KB requests

```
1M × 100 KB
≈ 100 GB/sec
≈ 800 Gbps
```

Now you're dealing with enormous network capacity.

Scenario C — HTTPS

If the LB terminates TLS, you also need to consider:

```
1M RPS
× TLS handshake/connection processing
× encryption/decryption
× HTTP parsing
× routing
```

So the actual sizing metric isn't merely:

> "Can the LB handle 1M RPS?"

You need:

```
RPS
+
bandwidth
+
connections/sec
+
concurrent connections
+
TLS handshakes/sec
+
CPU/request
+
number of routing rules
+
request/response size
```

---

2. Scale the LB horizontally

Suppose you've benchmarked one LB node and determined:

```
One LB node:
500K RPS
```

You could theoretically use:

```
1M / 500K = 2 nodes
```

But **you wouldn't deploy exactly two**.

You need redundancy and headroom.

For example:

```
LB-1 → 300K RPS
LB-2 → 300K RPS
LB-3 → 300K RPS
LB-4 → spare capacity
```

At normal operation:

```
Total traffic = 1M RPS

LB-1 = 250K
LB-2 = 250K
LB-3 = 250K
LB-4 = 250K
```

Now if one node disappears:

```
LB-1 ❌

Remaining:

LB-2 = ~333K
LB-3 = ~333K
LB-4 = ~333K
```

You have still maintained service.

This is the fundamental idea:

> **Provision enough LB capacity so that the remaining fleet can absorb traffic after the loss of one or more nodes.**

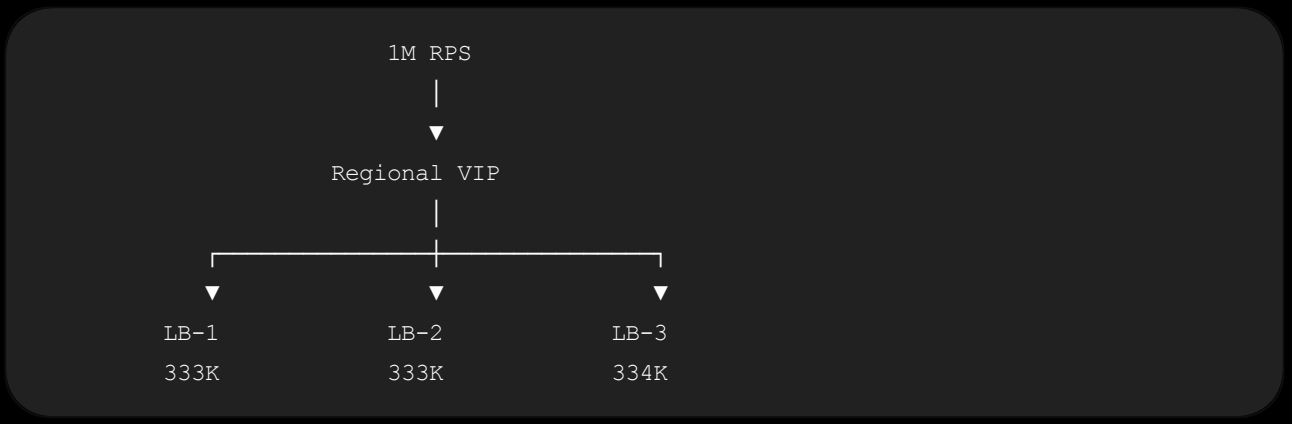
---

3. But who distributes 1M requests among LB nodes?

This is the interesting part.

You need a **front-end distribution mechanism**.

For example:



The VIP isn't necessarily a separate physical server.

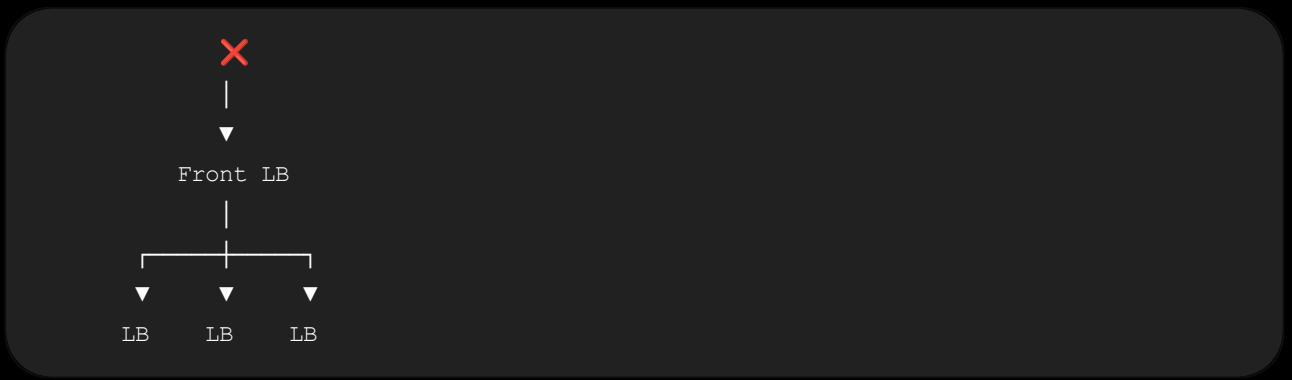
It can be implemented using:

- Anycast
- ECMP
- cloud-provider distributed frontend
- virtual IP
- network load balancing
- routing infrastructure

The crucial insight is:

> **Don't put another ordinary server in front of your load balancers.**

Otherwise you've simply moved the SPOF:



Now the "front LB" is the SPOF.

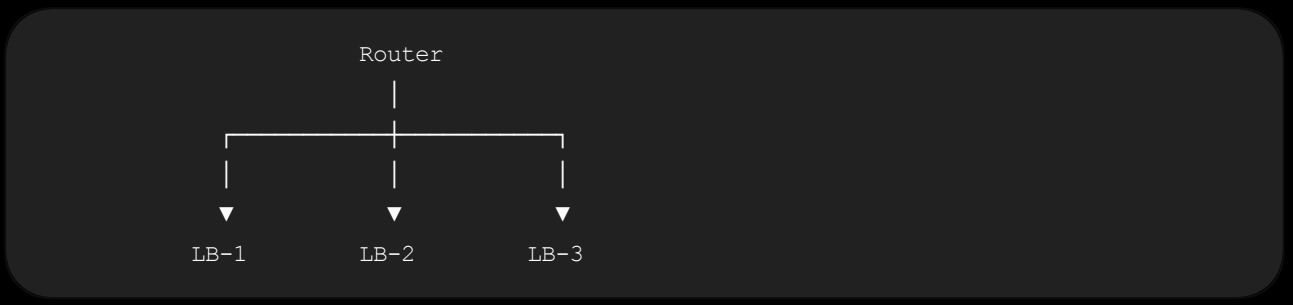
The entry layer itself must be distributed.

---

4. ECMP is particularly interesting at this scale

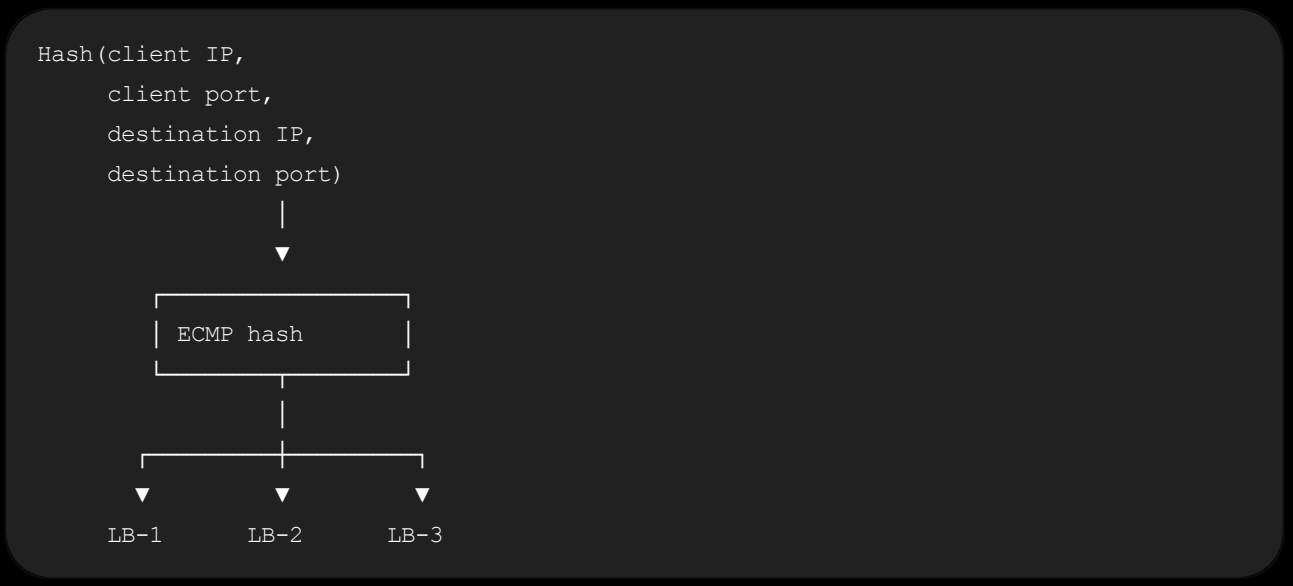
At high scale, the network can use **ECMP — Equal-Cost Multi-Path routing**.

Conceptually:



The router hashes flows across multiple paths.

For example:



This is extremely useful because you can scale the LB tier by adding more nodes.

---

5. Why hash flows instead of individual packets?

Suppose a single TCP connection contains:

```
Request 1
Request 2
Request 3
Request 4
```

You don't want:

```
Request 1 → LB-1
Request 2 → LB-2
```

```
Request 3 → LB-3
Request 4 → LB-1
```

That creates problems with connection state.

Instead:

```
TCP connection
      |
      ▼
   Flow hash
      |
      ▼
    LB-2
      |
      ├── Request 1
      ├── Request 2
      ├── Request 3
      └── Request 4
```

The flow remains associated with the same LB node.

---

6. HTTP/2 and HTTP/3 make this even more important

Consider:

```
Client
   |
   | one TCP connection
   ▼
   LB
   |
   ├── Request 1
   ├── Request 2
   ├── Request 3
   ├── Request 4
   └── Request 5
```

With HTTP/2, many requests can be multiplexed over one TCP connection.

With HTTP/3:

```
QUIC
  |
  └── multiple streams
```

So the LB generally needs to maintain the connection/session relationship.

This is one reason **connection distribution and request distribution are not necessarily the same thing**.

---

7. There are two fundamentally different scaling models

This distinction is excellent for interviews.

Model A — Scale by connections

```
Client
 |
 ▼
LB
 |
 ├── connection 1
 ├── connection 2
 ├── connection 3
 └── connection 4
```

The network distributes connections among LB nodes.

Model B — Scale requests

Suppose you have:

```
10,000 long-lived connections
```

but:

```
1M requests/sec
```

A connection might generate many requests.

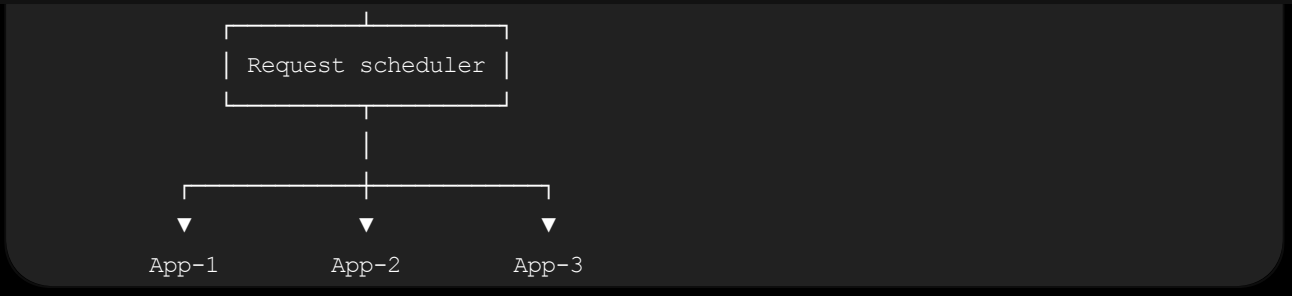
So:

```
Connections ≠ RPS
```

You may need to consider **request-level scheduling** inside the LB.

For example:

```
                LB cluster
                   |
```



---

8. Stateful vs stateless LB

This is another critical design decision.

Ideally, you want LB nodes to be as **stateless** as possible.

For example:

```
LB-1
LB-2
LB-3
LB-4
```

Any node should be capable of processing traffic.

If LB-1 stores critical state locally:

```
LB-1
 └── Session state
```

and LB-1 dies:

```
LB-1 ❌
     |
     └── Session state lost
```

You now have a failover problem.

Therefore large systems try to minimize state in the LB.

---

9. What about TLS?

This is one of the biggest practical considerations.

Suppose:

```
1M HTTPS requests/sec
```

If every request establishes a new TLS connection:

```
1M TLS handshakes/sec
```

That's very different from:

```
1M HTTP requests/sec
```

over persistent connections.

You therefore want:

```
Connection reuse
HTTP keep-alive
HTTP/2 multiplexing
TLS session resumption
```

to reduce handshake overhead.

The architecture might be:

```
Clients
   |
   ▼
TLS termination
   |
   ▼
LB cluster
   |
   ▼
HTTP/2 / HTTP
   |
   ▼
Application
```

Or TLS can be passed through if you need end-to-end encryption semantics:

```
Client
  | TLS
  ▼
L4 LB
  | TLS
  ▼
Application
```

That changes the LB's CPU profile considerably.

---

10. How do we decide how many LB nodes?

You **benchmark**, rather than assuming.

Suppose a realistic benchmark gives:

```
LB node:

CPU: 16 cores
Network: 25 Gbps

Performance:
    400K RPS
    2M concurrent connections
```

But you don't want to operate at 100%.

Suppose you target:

```
60% utilization
```

Then effective capacity is:

```
400K × 60%
= 240K RPS/node
```

For 1M RPS:

```
1,000,000 / 240,000
≈ 4.17
```

So you need at least:

```
5 nodes
```

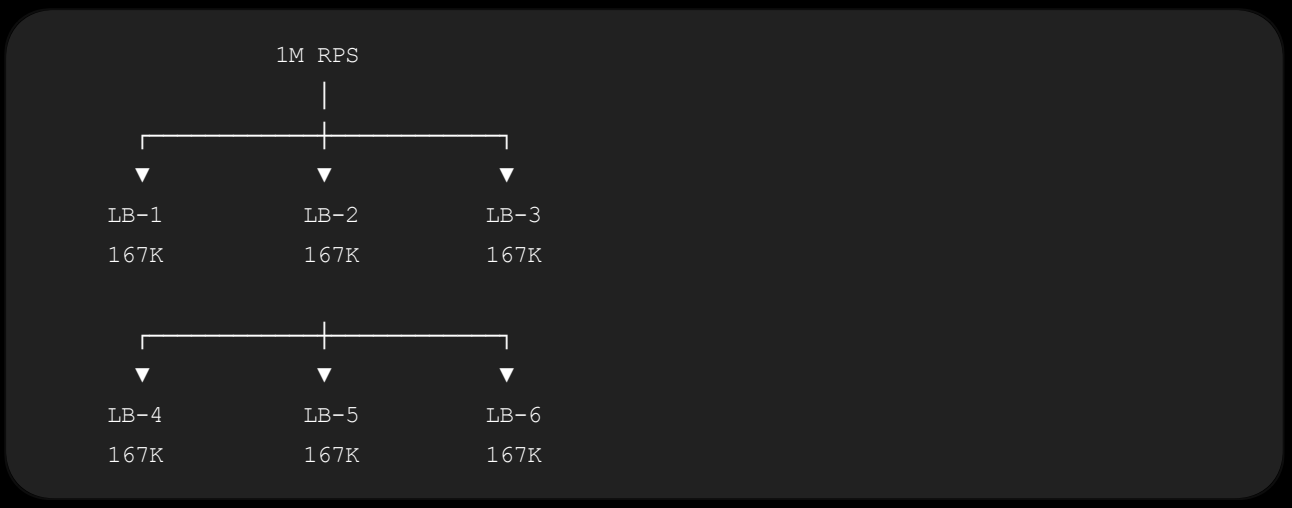
But that's still not necessarily sufficient.

You might provision:

```
6–8 nodes
```

depending on your failure requirement and growth/headroom strategy.

For example:



The fleet has substantial spare capacity.

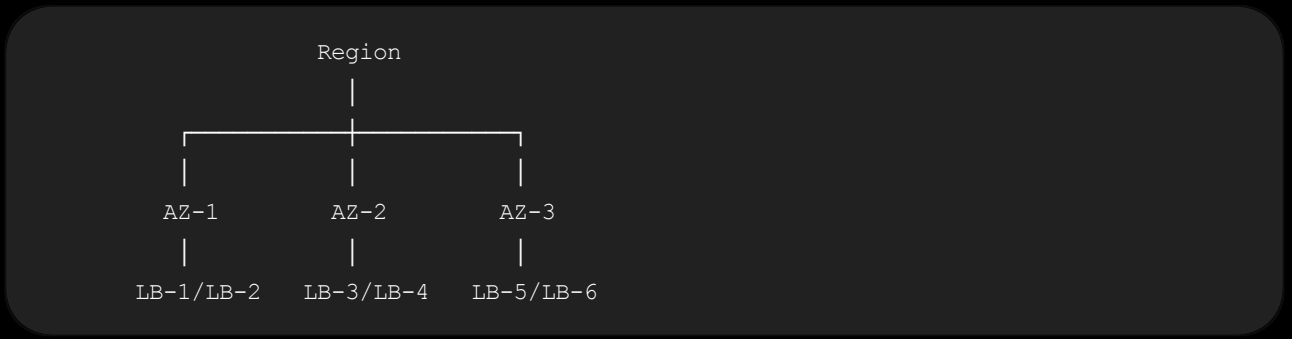
---

11. Distribute across Availability Zones

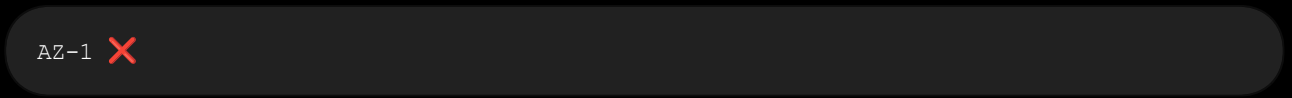
Never put all six here:



Instead:



Now:



leaves:

```
AZ-2 → LB-3/LB-4
AZ-3 → LB-5/LB-6
```

and the system continues operating.

---

12. What if one LB node fails?

Suppose:

```
LB-1  200K
LB-2  200K
LB-3  200K
LB-4  200K
LB-5  200K

Total = 1M
```

LB-3 fails:

```
LB-3 ❌
```

The traffic is redistributed:

```
LB-1 → 250K
LB-2 → 250K
LB-4 → 250K
LB-5 → 250K
```

This requires the front-end routing layer to detect/remove the failed node.

---

13. There's an important failure-mode problem

Suppose the network uses ECMP:

```
Router
 ├── LB-1
 ├── LB-2
 ├── LB-3
 └── LB-4
```

LB-3 dies.

The routing system needs to converge:

```
Before:

LB-1 ─┐
LB-2 ─┤
LB-3 ─┼── Traffic
LB-4 ─┘

After:

LB-1 ─┐
LB-2 ─┤
LB-4 ─┘── Traffic
```

This is why **failure detection and convergence time** matter.

A theoretically redundant architecture isn't enough.

You need to ask:

> How quickly can traffic stop being sent to the failed node?

---

14. One subtle problem: existing connections

Suppose:

```
Client
   |
   ▼
LB-3
   |
   ▼
Application
```

LB-3 suddenly dies.

Existing TCP connections attached to LB-3 may be lost.

Even if routing immediately redirects **new** connections:

```
New connections → LB-1/LB-2/LB-4
```

existing connections may still fail.

Therefore:

> **LB redundancy does not necessarily mean zero connection loss.**

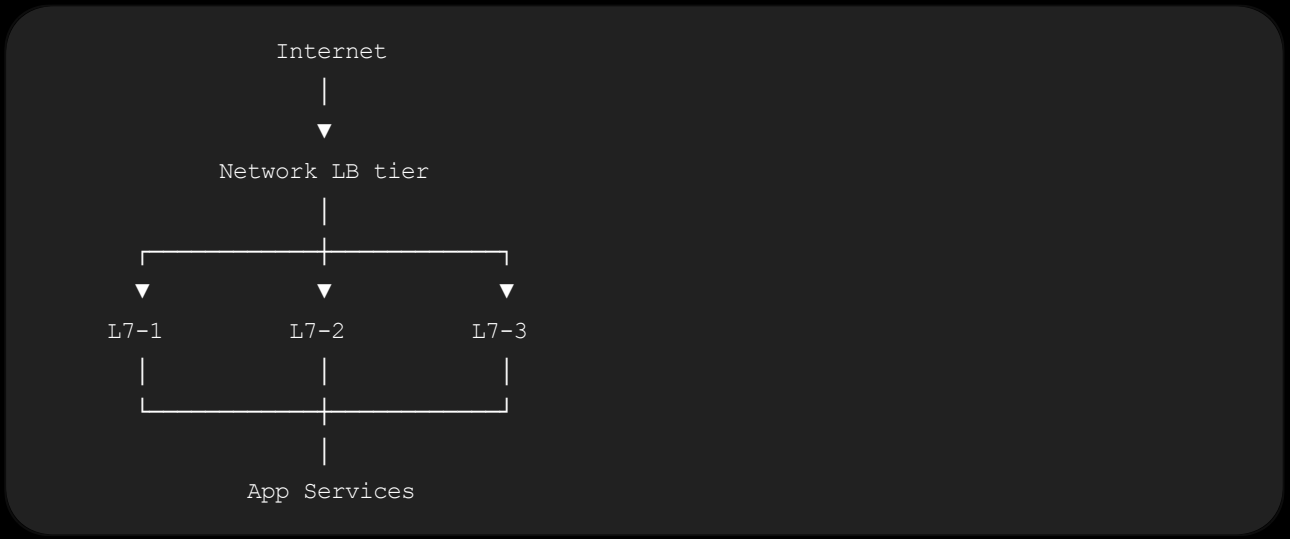
Applications should be designed for reconnection/retry where appropriate.

For WebSockets, MQTT, etc., this becomes especially important because connections are long-lived.

---

15. At 1M RPS, you may have another layer

A common architecture is:



The first layer handles things such as:

- IP/TCP
- connection distribution
- DDoS absorption
- network-level routing

The second layer handles:

- HTTP
- TLS
- URL routing
- headers
- authentication
- rate limiting
- service routing

This lets you scale different workloads independently.

---

16. And eventually the LB becomes a fleet

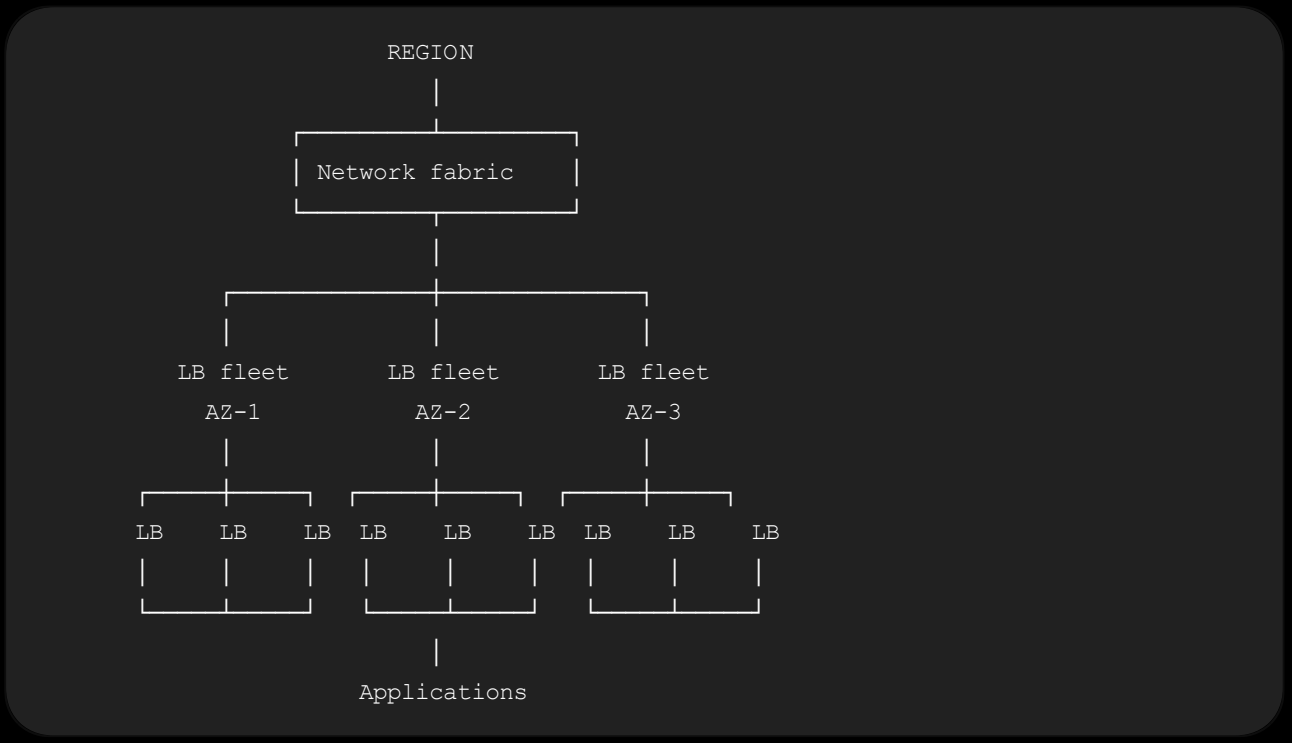
At sufficiently large scale, stop thinking:

> "Where is my load balancer?"

Start thinking:

> **"Where is my load-balancing fleet?"**

For example:



You can then autoscale the LB fleet based on:

```
RPS
CPU
network bandwidth
connections
TLS handshakes/sec
latency
queue depth
```

The most important interview insight

If the interviewer says:

> **"We have 1 million requests/sec going into one region. How do you scale the load balancer?"**

I'd structure the answer like this:

```
1. Don't use a single LB instance.
                ↓
2. Build an LB fleet.
                ↓
3. Distribute traffic using a distributed
   frontend: Anycast / ECMP / cloud LB frontend.
                ↓
4. Spread LB nodes across multiple AZs.
                ↓
5. Benchmark one node under realistic workload.
                ↓
6. Determine capacity based on:
       RPS
       bandwidth
       connections
       TLS
       request size
                ↓
7. Add N+1/N+2 capacity for failures.
                ↓
8. Health-check LB nodes and remove failures.
                ↓
9. Keep LB state minimal/stateless where possible.
                ↓
10. Design clients/applications to tolerate
    connection loss during LB failure.
```

And there's a **very important distinction**:

> **The load balancer is not necessarily the component doing all 1M RPS from one machine. The "LB" is a logical service backed by a horizontally scaled fleet of packet/request-processing nodes.**

That's exactly the same scalability principle you'll see with **Kubernetes control/data planes, Kafka brokers, RabbitMQ clusters, Cassandra nodes, WebSocket gateways, and MQTT brokers**: the thing we call "the service" is often actually a **distributed fleet**, with another distributed mechanism responsible for getting traffic to the fleet.